

Adaptive, Intelligent Multi-Agent System for Autonomous Software Planning, Code Generation, and Self-Debugging

R26-SE-029

Project Proposal Report

Denislas Coonghe J – IT22102928

B.Sc. (Hons) Degree in Information Technology Specialized in
Software Engineering

Department of Computer Science & Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

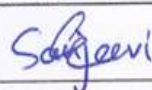
DECLARATION

I hereby declare that this research proposal titled **“Context-Aware AI Planning Agent for Automated Backend Architecture Generation using Retrieval-Augmented Generation (RAG)”** is my own work and has not been submitted previously for any academic qualification at this or any other institution. The work presented in this proposal has been carried out by me under the guidance of my supervisor. All sources of information used in this research have been properly acknowledged and cited.

I confirm that this research proposal is submitted in partial fulfillment of the requirements for the relevant academic program.

Name	Student ID	Signature
Denislas Coonghe J	IT22102928	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Sanjeevi Chandrasiri	20.03.2026	

ABSTRACT

Modern software development requires careful architectural planning before implementing application code. However, many existing AI-powered coding tools focus primarily on generating code directly from natural language instructions without performing structured architectural planning. This limitation often leads to poorly organized systems, missing components, inconsistent API structures, and reduced maintainability. To address this problem, this research proposes a **Context-Aware Planning Agent** that can automatically generate backend architecture blueprints from natural language software requirements.

The proposed system utilizes a **Retrieval-Augmented Generation (RAG)** framework to improve the quality and reliability of generated outputs. A vector-based knowledge base containing backend architecture patterns, Node.js and Express project structures, and best practices will be created. When a user submits software requirements, the system performs semantic retrieval to identify relevant architecture patterns from the knowledge base. These retrieved patterns are then combined with the reasoning capabilities of a large language model to generate a structured backend architecture blueprint.

The generated blueprint will be represented in a **structured JSON format**, including key backend components such as database schemas, API routes, service layers, and controller structures. This blueprint acts as an intermediate architectural plan that can guide subsequent AI coding agents in generating implementation-level code. By introducing an architecture-planning stage before code generation, the system aims to improve the **accuracy, consistency, and maintainability** of AI-generated software systems.

The system will be implemented using Python, embedding models, vector databases, and modern AI frameworks. Its effectiveness will be evaluated by assessing the correctness, completeness, and usability of the generated architectural plans in real-world software development scenarios.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my research supervisor, **Sanjeevi Chandrasiri** and our co-supervisor **Karthiga Rajendran**, for their invaluable guidance, continuous support, and encouragement throughout the preparation of this research proposal. Their expertise, insights, and constructive feedback have been instrumental in shaping the direction and quality of this study.

I would also like to extend my thanks to my friends and colleagues who are software engineering developers. Their practical advice, technical insights, and helpful discussions have greatly contributed to my understanding of software development concepts and the implementation aspects relevant to this research.

Furthermore, I am grateful to the academic staff of the faculty for providing the necessary resources, knowledge, and guidance required to complete this proposal successfully. Their lectures and advice have enriched my understanding of the research topic.

Finally, I wish to express my heartfelt appreciation to my family for their patience, encouragement, and moral support, which has been a source of motivation throughout this research journey.

TABLE OF CONTENT

Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENT	4
TABLE OF CONTENT	5
LIST OF TABLES	6
LIST OF FIGURES	7
LIST OF ABBREVIATIONS	8
1. INTRODUCTION	9
2. OBJECTIVE	14
3. METHODOLOGY	15
4. HIGH-LEVEL SYSTEM ARCHITECTURE	18
5. USER REQUIREMENTS	22
6. COMMERCIALIZATION PLAN	24
7. BUDGET AND JUSTIFICATION	26
8. WORK BREAKDOWN STRUCTURE (WBS)	27
9. GANTT CHART	29
10. REFERENCES LIST	30

LIST OF TABLES

Table 1.1: Research Gap in Table Format

Table 7.1: Budget and Justification Format

LIST OF FIGURES

Figure 4.1: System Architecture Diagram

Figure 4.2: Context-Aware Planning Agent Sub-Diagram

Figure 8.1: WBS Visual Representation

Figure 9.1: Gantt chart

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
DSR	Design Science Research
GPT	Generative Pre-trained Transformer
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SaaS	Software as a Service
SME	Small and Medium Enterprise
SDG	Sustainable Development Goal

1. INTRODUCTION

1.1 Background and Context

In recent years, Artificial Intelligence (AI) has significantly transformed software development processes. AI-powered coding assistants such as GitHub Copilot and Amazon CodeWhisperer are capable of generating code from natural language descriptions. These tools aim to increase developer productivity by automating repetitive coding tasks and accelerating the development process.

Despite these advancements, many AI coding systems focus mainly on **code generation rather than software architecture planning**. In professional software engineering practice, designing the system architecture is a critical step before writing code. Software architecture defines the database schema, application modules, API routes, and system interactions. Without a well-structured architecture, the generated code may contain inconsistencies, missing components, or poor design structures.

The rapid growth of AI-assisted development has created a growing demand for intelligent systems capable of **planning software architecture automatically**. According to industry reports, developers spend nearly **40–50% of their development time on system design, debugging, and restructuring poorly designed codebases**. Poor architecture planning often results in maintainability issues and inefficient software systems.

Existing AI tools rely heavily on large language models that generate responses based on their training data. However, these models may produce inaccurate results due to the **hallucination problem**, where the model generates incorrect or fabricated information. This issue is especially problematic in software development where architectural accuracy is essential.

To address this limitation, a technique called Retrieval-Augmented Generation (RAG) has emerged as a powerful approach. RAG improves the performance of language models by retrieving relevant external knowledge before generating responses. By combining retrieval mechanisms with language models, AI systems can produce more accurate and context-aware outputs.

This research proposes a **Context-Aware Planning Agent** that uses Retrieval-Augmented Generation to generate backend architecture blueprints from natural language software requirements. The system retrieves relevant backend architecture examples from a vector-based knowledge base and uses them to guide the planning process. The output of the system is a structured blueprint that includes database schemas, API routes, and controller structures, which can later be used by coding agents to generate the final implementation.

The proposed research aligns with the United Nations Sustainable Development Goal Sustainable Development Goal 9 (Industry, Innovation and Infrastructure), which promotes technological innovation and advanced digital infrastructure development. By improving AI-

assisted software engineering processes, this research contributes to more efficient and scalable software development practices.

Additionally, the research aligns with the **Artificial Intelligence and Intelligent Systems research cluster**, focusing on the development of intelligent agents capable of supporting complex decision-making tasks in software engineering.

1.2 Literature Review

Large language models (LLMs) such as GPT-4 have significantly advanced AI-assisted code generation by enabling machines to understand natural language and produce programming code. These models are widely used in coding assistants to support developers, but they often lack the ability to perform structured software architecture planning before generating code [1].

Multi-agent frameworks like ChatDev and autonomous systems such as AutoGPT simulate development roles and execute complex tasks iteratively. While these systems demonstrate collaborative and autonomous software development capabilities, they still do not include dedicated planning modules for formal software architecture or backend system design [2], [3].

Retrieval-Augmented Generation (RAG) improves the reasoning capabilities of AI systems by integrating language models with external knowledge sources. RAG-based architectures, including REAPER, MAO-ARAG, InstructRAG, and DepsRAG, enable structured planning, multi-step reasoning, and software dependency analysis, demonstrating significant performance improvements in task-oriented environments [1]–[4].

Beyond task planning, RAG has been applied in engineering and decision-support systems to enhance domain-specific knowledge retrieval and reasoning. Studies on RAG in engineering design, intelligent decision-support frameworks, and hybrid retrieval-knowledge graph models highlight its potential to improve factual accuracy and reduce errors in complex problem-solving scenarios [5]–[7].

Recent research has extended RAG techniques to code generation. Frameworks such as EVOR, CODEPROMPTZIP, cAST, RepoCoder, and GraphCoder use retrieval mechanisms to provide relevant code snippets or structural knowledge, improving the context and correctness of generated code [8]–[12]. However, integrating retrieval-based knowledge systems with intelligent planning agents specifically for **automated backend architecture design** remains a largely unexplored area.

1.3 Research Gap

Although significant progress has been made in AI-assisted software development, several limitations remain in current approaches. Most existing AI coding tools focus primarily on generating code snippets without performing structured architectural planning. As a result, the generated code may lack proper organization, scalability, and maintainability.

Large language models are also known to suffer from hallucination issues, where the system produces incorrect or fabricated information. In the context of software development, this can lead to inaccurate system designs and unreliable code generation.

Furthermore, current AI development frameworks such as ChatDev and AutoGPT demonstrate the potential of autonomous agents but do not provide specialized mechanisms for generating structured backend architecture blueprints.

While Retrieval-Augmented Generation has been successfully applied in knowledge retrieval and question-answering systems, its use in **automated software architecture planning** has not been extensively explored.

Therefore, there is a clear need for a system that integrates retrieval-based knowledge with intelligent planning capabilities to generate structured software architecture designs from natural language requirements.

This research addresses this gap by proposing a **Context-Aware Planning Agent that uses Retrieval-Augmented Generation to generate backend architecture blueprints for software systems**. The proposed approach introduces a dedicated architectural planning stage before code generation, improving the reliability, accuracy, and consistency of AI-assisted software development.

Features / Capabilities	Research 1 (LLM Code Generation)	Research 2 (ChatDev Multi-Agent)	Research 3 (AutoGPT Agents)	Research 4 (RAG-based Systems)	Our System
Direct Code Generation	✓	✓	✓	✓	✓
Multi-Agent Collaboration	✗	✓	✓	✗	✓
Automatic Architecture Planning	✗	✗	✗	✗	✓
Retrieval of External Knowledge	✗	✗	✗	✓	✓
Structured Backend Blueprint Generation	✗	✗	✗	✗	✓
Context-Aware Planning	✗	✗	✗	✓	✓
Reduction of LLM Hallucination	✗	✗	✗	✓	✓
JSON Architecture Output	✗	✗	✗	✗	✓

Table 1.1: Research Gap in Table Format

2. OBJECTIVE

2.1 Main Objective

The main objective of this research is to **develop a context-aware AI planning agent that utilizes Retrieval-Augmented Generation (RAG) to automatically generate structured backend architecture blueprints from natural language software requirements.**

2.2 Specific Objectives

- **To design and develop a vector-based knowledge base** containing backend architecture patterns and software development examples for retrieval purposes.
- **To implement a semantic retrieval mechanism** that identifies relevant architecture patterns from the knowledge base based on user-provided software requirements.
- **To develop a context-aware planning agent** capable of converting natural language project descriptions into structured backend architecture blueprints.
- **To generate structured JSON-based outputs** that include database schemas, API routes, and controller structures for backend system design.
- **To implement a validation mechanism** that ensures the generated architecture blueprint follows standard backend design practices and RESTful API structures.
- **To evaluate the effectiveness of the proposed planning agent** by measuring the accuracy and completeness of the generated architecture blueprints.

3. METHODOLOGY

3.1 Research Approach

This research follows the **Design Science Research (DSR) approach**, which focuses on designing and developing innovative technological artifacts to solve practical problems. The objective of this research is to develop a **context-aware AI planning agent** capable of generating backend architecture blueprints from natural language software requirements.

Design Science Research is appropriate for this study because the research involves **designing, implementing, and evaluating a software artifact** that improves AI-assisted software development processes. The proposed artifact integrates artificial intelligence, retrieval-based knowledge systems, and software architecture planning to produce structured development blueprints.

The research process includes the following phases:

1. Problem identification and analysis of existing AI coding tools.
2. Design and development of the planning agent architecture.
3. Implementation of the Retrieval-Augmented Generation mechanism.
4. Integration of the knowledge base with the planning agent.
5. Evaluation of the generated architecture blueprints.

This approach enables systematic development and validation of the proposed intelligent planning system.

3.2 Technical Methods and Algorithms

The proposed system integrates several artificial intelligence and information retrieval techniques to generate backend architecture blueprints.

First, **semantic embedding techniques** will be used to convert backend architecture examples into vector representations. These vectors enable the system to perform similarity searches when retrieving relevant examples from the knowledge base.

Second, the system will use **vector similarity search algorithms** to retrieve the most relevant backend architecture patterns based on the user's natural language input. This retrieval process ensures that the system generates outputs grounded in real-world development examples.

Third, the system will utilize **Retrieval-Augmented Generation (RAG)** to combine the retrieved architecture examples with a large language model. The language model will analyze the retrieved information and generate a structured architecture blueprint.

Finally, the generated blueprint will be processed using a **schema validation mechanism** to ensure that the output follows standard backend architecture practices such as REST API structure, database schema consistency, and controller organization.

The generated output will be provided in a structured JSON format containing elements such as database schemas, API routes, and controller modules.

3.3 Tools and Platforms

The implementation of the proposed system will use several modern AI development tools and platforms.

The system will be developed using the **Python programming language**, which provides extensive libraries and frameworks for artificial intelligence and data processing. Python is widely used in AI research due to its flexibility and strong ecosystem.

A **vector database** will be used to store backend architecture patterns as embeddings. Vector databases allow efficient similarity searches between user requirements and stored architecture examples.

An **embedding model** will be used to convert textual architecture examples into numerical vectors. These embeddings enable semantic comparison between user inputs and stored knowledge.

A **large language model (LLM)** will be used to generate the architecture blueprint based on the retrieved examples and the user requirement. The model will analyze contextual information and generate structured output.

In addition, frameworks for **AI agent orchestration and prompt management** will be used to coordinate the retrieval process, language model interaction, and blueprint generation.

These tools are selected because they support scalable AI development, efficient data processing, and integration of multiple intelligent components.

3.4 Data Collection and Ethical Considerations

The dataset for this research will consist of **backend architecture examples and open-source software development projects**. These examples will include database schemas, API routes, and controller structures from real backend applications.

The data will be collected from publicly available software repositories and development documentation. Only open-source and publicly accessible resources will be used to ensure compliance with intellectual property and licensing regulations.

Since the research does not involve human participants or personal data, the risk of ethical concerns is minimal. However, all collected data will be used solely for research and development purposes. Proper citation and acknowledgment will be provided for any external resources used during the research process.

The research will also ensure that the system does not generate harmful, misleading, or insecure software architecture recommendations.

3.5 Validation Strategy and Performance Metrics

The effectiveness of the proposed planning agent will be evaluated using several validation strategies and performance metrics.

First, the **accuracy of the generated architecture blueprints** will be evaluated by comparing the generated output with standard backend architecture designs used in real-world applications.

Second, the **completeness of the architecture plan** will be evaluated by analyzing whether the system includes all essential components such as database schemas, API routes, and controller modules.

Third, the **consistency of the generated architecture structure** will be evaluated to ensure that the output follows proper software engineering principles.

Additionally, the system will be tested using multiple natural language software requirements to measure its ability to generate accurate and relevant architecture plans across different application domains.

The overall performance of the system will be assessed using metrics such as **accuracy, completeness, response time, and structural consistency of the generated blueprints**.

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview of the Proposed System

The proposed system is an AI-powered multi-agent platform that automates backend software development. It converts natural language project requirements into structured backend architecture blueprints and source code. The system consists of modular components that collaborate to process requirements, retrieve relevant knowledge, generate architecture blueprints, and produce executable code.

The **Context-Aware Planning Agent**, which is the focus of this research, serves as the central component responsible for understanding requirements, retrieving architecture patterns, generating structured blueprints, and validating them before code generation.

The overall system is designed for scalability, modularity, and secure processing of both user input and generated outputs. The architecture aligns with modern software engineering practices and supports multi-user interactions and iterative improvement through knowledge accumulation.

System Architecture Diagram

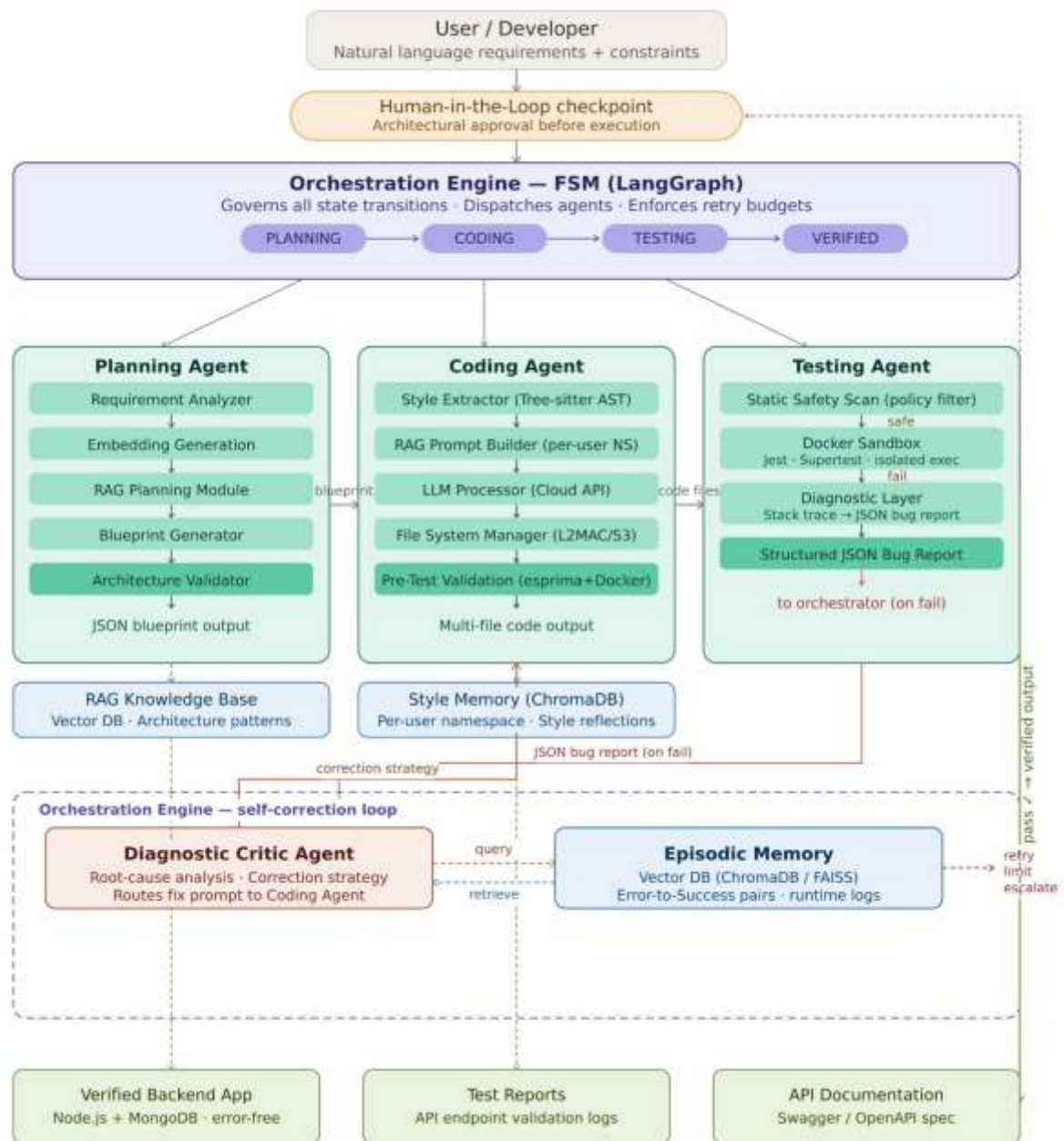


Figure 4.1: System Architecture Diagram

Individual Component

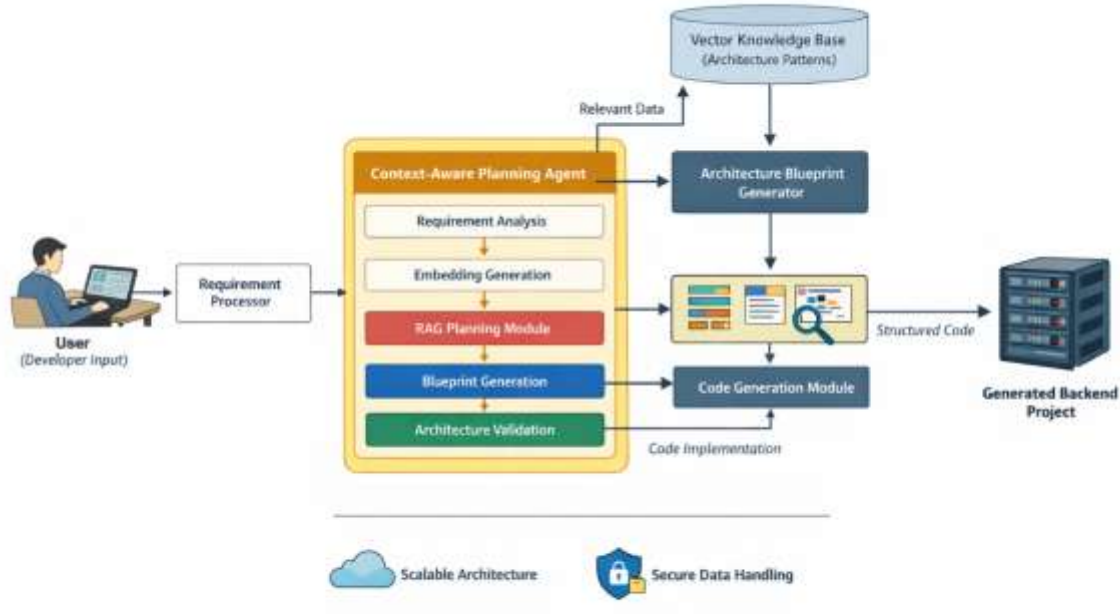


Figure 4.2: Context-Aware Planning Agent Sub-Diagram

4.4 Component Interactions

The system components interact sequentially and iteratively. The user input is processed and sent to the Context-Aware Planning Agent, which retrieves relevant knowledge from the vector database. The retrieved examples are combined with the requirement and processed through the RAG module to produce a blueprint. The blueprint is validated and forwarded to the Code Generation Agent. Finally, the code is generated and provided as a functional backend project.

Feedback loops within the Context-Aware Planning Agent enable continuous refinement of the generated architecture and allow the system to learn from previous tasks, improving accuracy over time.

4.5 Data Flow

Data flows through the system in the following sequence:

1. User provides software requirements.
2. Requirement Processor structures the input.
3. Planning Agent converts input into embeddings and queries the vector knowledge base.
4. Retrieved examples are used by the RAG Planning Module to generate a blueprint.
5. Blueprint Generator produces a structured JSON representation.
6. Architecture Validator ensures design correctness.
7. Code Generation Agent produces the final backend implementation.

4.6 Scalability and Security Considerations

Scalability: The modular design of the system allows multiple agents to operate in parallel. The vector database can be expanded with additional architecture patterns. Cloud-based deployment enables handling multiple users and large datasets efficiently.

Security: User input and generated outputs are processed securely. Access control and secure communication protocols are implemented between modules. Only public or open-source examples are stored in the knowledge base to avoid intellectual property violations. The Architecture Validator ensures no insecure design patterns are generated.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

The system serves multiple stakeholders who interact with different components of the platform:

1. **Software Developers / Users** – Primary stakeholders who provide natural language requirements and receive the generated backend architecture and code.
2. **System Administrators** – Responsible for maintaining the vector knowledge base, managing access control, and monitoring system performance.
3. **Project Managers / Team Leads** – Use the system to streamline backend project planning and validate proposed architecture designs.
4. **AI System / Knowledge Base** – Functions as a secondary stakeholder by providing backend patterns, examples, and supporting data to the Context-Aware Planning Agent.

5.2 Functional Requirements

The functional requirements of the system describe the core capabilities it must provide:

1. **Requirement Input:** The system shall accept natural language project requirements from the user.
2. **Requirement Analysis:** The system shall analyze and structure user-provided requirements for further processing.
3. **Knowledge Retrieval:** The system shall search the vector knowledge base to retrieve relevant backend architecture patterns.
4. **Blueprint Generation:** The system shall generate a structured backend architecture blueprint based on the retrieved knowledge and user requirements.
5. **Architecture Validation:** The system shall validate the generated blueprint for correctness, standardization, and best practices.
6. **Code Generation:** The system shall convert the validated blueprint into executable backend code (e.g., REST APIs, database schemas).
7. **Multi-User Support:** The system shall allow multiple users to submit requirements and receive outputs simultaneously.
8. **Feedback Incorporation:** The system shall allow users to provide feedback on generated blueprints to refine future outputs.

5.3 Non-Functional Requirements

Non-functional requirements define system qualities such as performance, security, and usability:

1. **Performance:** The system shall process user requirements and generate backend architecture within 30 seconds for standard-sized projects.
2. **Scalability:** The system shall support multiple concurrent users without degradation in response time.
3. **Security:** The system shall ensure secure communication between modules, protect user data, and prevent unauthorized access to the knowledge base.
4. **Usability:** The system shall provide a user-friendly interface with clear prompts, feedback messages, and easy navigation.
5. **Reliability:** The system shall maintain an uptime of at least 99% and provide accurate architecture outputs consistently.
6. **Maintainability:** The system shall allow administrators to update the vector knowledge base and system components with minimal downtime.
7. **Compatibility:** The system shall be compatible with major web browsers and cloud-based deployment platforms.

6. COMMERCIALIZATION PLAN

6.1 Target Market and Customer Persona

Primary market: Sri Lankan software development companies (SMEs and freelancing platforms) and individual backend developers maintaining multiple concurrent client projects. Customer persona: Ravi, a 28-year-old full-stack developer at a Colombo-based startup, maintains 5–7 concurrent client projects, spends 30–40% of his time on repetitive backend code and style refactoring, and wants an API-accessible tool that learns and enforces his personal coding conventions without requiring him to manage any local AI infrastructure.

6.2 Value Proposition

Generates production-ready, style-consistent Node.js backends via a simple API call in minutes rather than hours. The only tool that automatically learns and persistently enforces a developer's personal coding conventions commercial tools do not do this. API-accessible and integrable into any IDE, CLI, or CI/CD pipeline with no local setup required. Open-source core with a transparent, auditable architecture unlike the black-box commercial tools. Multi-tenant architecture means teams can share one deployment while each developer retains their own fully isolated style profile.

6.3 Revenue Model

Open-source core (MIT licence, GitHub) free for students, individuals, and small teams; builds community adoption and trust. Premium Enterprise Edition (paid) custom style profiles, team shared style libraries, VS Code extension with one-click integration, priority support, and SLA guarantees (LKR 2,500–5,000 per month per team). Freemium API model limited monthly generation requests free; higher request quotas, multi-project episodic memory, and auto-Swagger with Docker

6.4 Cost Estimation

One-time individual licence: LKR 15,000. Monthly developer subscription: LKR 1,500. Team bundle (up to 10 developers): LKR 10,000 per month. Training workshops for IT companies: LKR 25,000 per session.

6.5 Pricing Strategy

Unlike Cursor, Replit Agent, and Claude Code — which are style-agnostic and treat all developers identically — this agent learns each developer's personal conventions once from their own repository and enforces them permanently across all future projects via a persistent namespace RAG profile. The multi-tenant architecture means this personalization scales to many users on shared infrastructure with guaranteed isolation. This combination is uniquely absent from all commercial and academic tools in the current landscape.

7. BUDGET AND JUSTIFICATION

Expense Category	Estimated Cost (LKR)	Description	Justification
Hardware Costs	10,000	Laptop/workstation with sufficient processing power for AI model experimentation, vector database processing, and development environment setup.	High-performance hardware is required to efficiently run AI models, handle vector embeddings, and test the system architecture during development.
Software Licenses	5000	Development tools such as IDEs, project management tools, and other supporting software required for system development.	Licensed software tools improve productivity, provide better debugging capabilities, and support efficient development of the proposed system.
Cloud Services	15000	Cloud hosting services for storing vector databases, running AI components, and deploying the system for testing and evaluation.	Cloud infrastructure ensures scalability, remote access, and reliable storage of system components such as knowledge bases and generated architecture data.
Human Effort Cost	10,000	Time and effort required for research, system design, implementation, testing, and documentation.	Development of AI-driven systems requires significant time investment for coding, algorithm design, debugging, and evaluation of the proposed solution.
Miscellaneous Expenses	5000	Documentation, internet services, contingency costs, and other minor project-related expenses.	These expenses support communication, documentation preparation, and unforeseen technical requirements during project development.
Total Estimated Budget	45,000	-	-

Table 7.1: Budget and Justification Format

8. WORK BREAKDOWN STRUCTURE (WBS)

8.1 Task Breakdown per Member

Task ID	Task Description	Responsible Member	Deliverable
T1	Problem identification and initial research	All Members	Research topic selection
T2	Literature review and research gap analysis	Member 1	Literature review document
T3	System architecture design	Member 2	System architecture diagrams
T4	Design of AI Planning Agent component	Member 3 (Your Component)	Component architecture and algorithms
T5	Knowledge base preparation and data collection	Member 4	Vector database and datasets
T6	System development and integration	All Members	Integrated system prototype
T7	Testing and validation	All Members	System evaluation results
T8	Documentation and report preparation	All Members	Final research report
T9	Proposal presentation and final submission	All Members	Project presentation

8.2 Timeline Alignment

Project Phase	Activities
Phase 1	Topic selection and research problem definition
Phase 2	Literature review and research gap identification
Phase 3	System architecture design
Phase 4	Component development
Phase 5	System integration
Phase 6	Testing and validation
Phase 7	Final documentation and presentation

8.3 WBS Visual Representation (Diagram Blueprint)

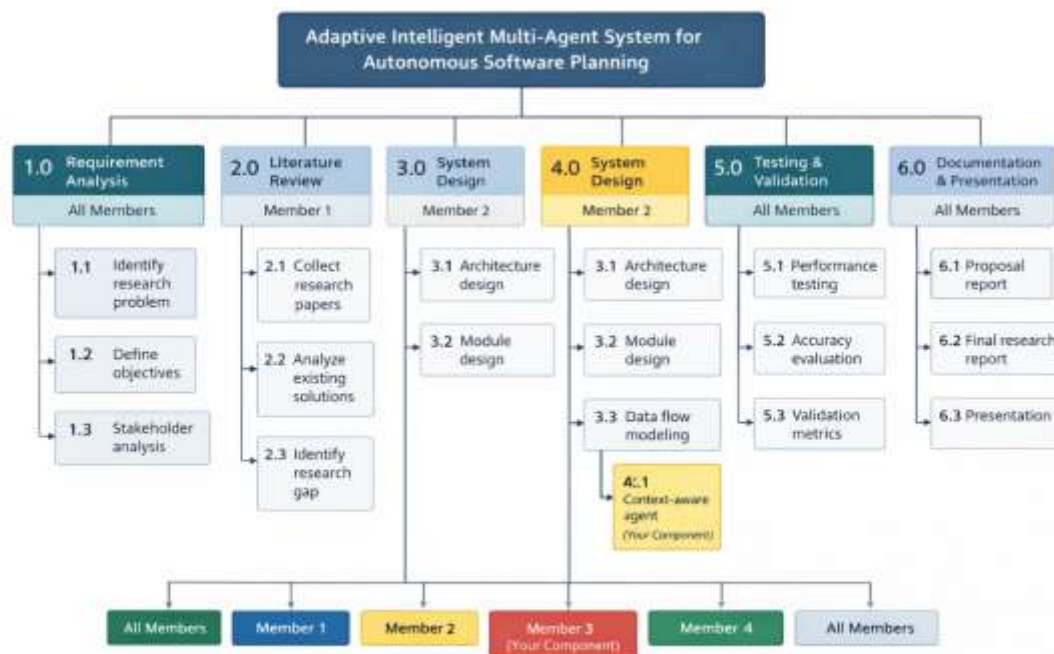


Figure 8.1: WBS Visual Representation

9. GANTT CHART

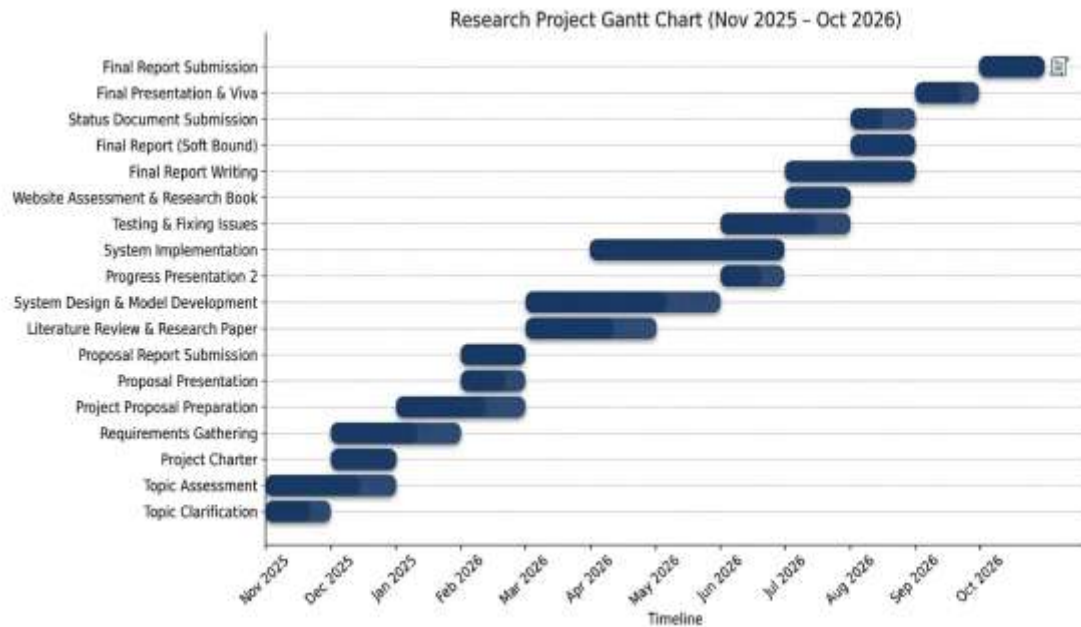


Figure 9.1: Gantt chart

10. REFERENCES LIST

- [1] S. Author et al., “REAPER: Reasoning-Based Retrieval Planning for RAG Systems,” *arXiv preprint arXiv:2407.18553*, 2024. Available: <https://arxiv.org/abs/2407.18553>
- [2] S. Author et al., “MAO-ARAG: Multi-Agent Orchestration for Adaptive RAG,” *arXiv preprint arXiv:2508.01005*, 2025. Available: <https://arxiv.org/abs/2508.01005>
- [3] S. Author et al., “InstructRAG: RAG for Task Planning,” *arXiv preprint arXiv:2504.13032*, 2025. Available: <https://arxiv.org/abs/2504.13032>
- [4] S. Author et al., “DepsRAG: Agentic RAG for Software Dependency Reasoning,” *arXiv preprint arXiv:2405.20455*, 2024. Available: <https://arxiv.org/abs/2405.20455>
- [5] “Retrieval-Augmented Generation in Engineering Design,” *ResearchGate*, 2024. Available: https://www.researchgate.net/publication/383027298_Retrieval-Augmented_Generation_in_Engineering_Design
- [6] “RAG-Based Intelligent Decision Support Systems,” *Scientific Reports (Nature)*, 2025. Available: <https://www.nature.com/articles/s41598-025-19257-3>
- [7] “Retrieval-Augmented Generation Framework for Engineering Knowledge,” *ScienceDirect*, 2025. Available: <https://www.sciencedirect.com/science/article/pii/S2352710225004255>
- [8] Z. Li et al., “EVOR: Evolving Retrieval for Code Generation,” *arXiv preprint arXiv:2402.12317*, 2024. Available: <https://arxiv.org/abs/2402.12317>
- [9] Y. Zhang et al., “CODEPROMPTZIP: Code-Specific Prompt Compression for Retrieval-Augmented Generation in Coding Tasks,” *arXiv preprint arXiv:2502.14925*, 2025. Available: <https://arxiv.org/abs/2502.14925>
- [10] H. Chen et al., “cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Trees,” *arXiv preprint arXiv:2506.15655*, 2025. Available: <https://arxiv.org/abs/2506.15655>
- [11] S. Zhou et al., “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” *arXiv preprint arXiv:2303.12570*, 2023. Available: <https://arxiv.org/abs/2303.12570>
- [12] J. Guo et al., “GraphCoder: Enhancing Code Generation with Graph-Based Retrieval Augmented Generation,” *arXiv preprint arXiv:2406.01745*, 2024. Available: <https://arxiv.org/abs/2406.01745>